# Table creation

```
CREATE TABLE table_name
 (
    column1 datatype [ NULL | NOT NULL ],
    column2 datatype [ NULL | NOT NULL ],
    ...
     column_n datatype [ NULL | NOT NULL ]
);
```

# Example

```
CREATE TABLE customers1
 (
   customer_id int(10) NOT NULL,
  customer_name varchar(50) NOT NULL,
   city varchar(50)
);
```

# UNIQUE constraints

- A unique constraint is a single field or combination of fields that uniquely defines a record.
- Some of the fields can contain null values as long as the combination of values is unique.
- We may define more than one **UNIQUE constraints**

# UNIQUE constraints

```
CREATE TABLE table_name
(
    ...
    column_name  data_type UNIQUE
    ...
);
```

# UNIQUE constraints

```
CREATE TABLE table_name (
    ...
    column_name1 data_type,
    column_name2 data_type,
    ...,
    CONSTRAINT unique_constraint_name UNIQUE
    (column_name1, column_name2)
);
```

# What is a primary key

- A **primary key** is a single field or combination of fields that uniquely defines a record.
- **None of the fields** that are part of the primary key **can contain** a **null value**.
- A table can have only **one primary key**.

# Table creation with primary key

```
CREATE TABLE table_name
 (
column1 datatype primary key,
column2 datatype null/not null,
 ...
);
```

# Table creation with primary key

```
CREATE TABLE table_name
 (
column1 datatype null/not null,
column2 datatype null/not null,
 ...
primary key (column1, column2)
);
```

# Table creation with primary key

```
CREATE TABLE table_name
 (
column1 datatype null/not null,
column2 datatype null/not null,
 ...
 CONSTRAINT constraint_name PRIMARY KEY
    (column1, column2, ... column_n)
);
```

# Example

```
CREATE TABLE supplier
 (
 supplier_id int(10) primary key,
 supplier_name varchar(50) not null,
 contact_name varchar(50)
);
```

# Example

```
CREATE TABLE supplier1
 (
 supplier_id int(10) ,
 supplier_name varchar(50) not null,
 contact_name varchar(50),
 PRIMARY KEY(supplier_id)
);
```

# Primary key with more than one field

```
CREATE TABLE supplier2
 (
    supplier_id int(10) not null,
   supplier_name varchar(50) not null, contact_name
    varchar(50),
    PRIMARY KEY (supplier_id , supplier_name )
);
```

# Foreign Keys

- A foreign key is a way to enforce referential integrity within database.
- A foreign key means that values in one table must also appear in another table.
- The referenced table is called the *parent table* while the table with the foreign key is called the *child table*.
- The foreign key in the child table will generally reference a primary key in the parent table.

# Foreign Keys

```
CREATE TABLE table_name
(
column1 datatype null/not null,
 column2 datatype null/not null,
 ...
CONSTRAINT fk_column FOREIGN KEY
(column1, column2, ... column_n)
REFERENCES parent_table
 (column1, column2, ... column_n)
 );
```

# Foreign key

```
CREATE TABLE supplier3
 (
supplier_id int(10) not null,
supplier_name varchar(50) not null,
 contact_name varchar(50),
 PRIMARY KEY (supplier_id)
 );
```

# Foreign key

```
CREATE TABLE products3
 (
 product_id int(10) not null,
 supplier_id int(10) not null,
 FOREIGN KEY (supplier_id) REFERENCES
    supplier3(supplier_id)
);
```

# Check Constraint

A **check constraint** allows us to specify a condition on each row in a table.

# Check Constraint

```
CREATE TABLE table_name
 (
 column1 datatype null/not null,
 column2 datatype null/not null,
 ...
CONSTRAINT constraint_name CHECK (column_name
    condition) [DISABLE]
);
```

# Example 1

```
CREATE TABLE suppliers
(
supplier_id int(4),
supplier_name varchar(50),
CHECK (supplier_id BETWEEN 100 and 9999)
);
```

# Example 1

```
CREATE TABLE suppliers
(
supplier_id int(4),
supplier_name varchar(50),
CONSTRAINT check_supplier_id CHECK
    (supplier_id BETWEEN 100 and 9999)
);
```

# Example 2

```
CREATE TABLE Persons1
(
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int ,CHECK (Age>=18)
);
```

# ALTER TABLE statement

**ALTER TABLE statement** to

add a column,

modify a column,

drop a column,

rename a column or

rename a table

# Add one or more columns

```
ALTER TABLE table_name
ADD column_name      column_definition;

    ALTER TABLE table_name
    ADD
    (
    column_1             column_definition,
    column_2             column_definition,
    ...
    column_n             column_definition
    );
```

# Modify column in table

```
ALTER TABLE table_name
MODIFY column_name column_type;

ALTER TABLE table_name
MODIFY
 (
column_1 column_type,
column_2 column_type,
...
column_n column_type);
```

# Rename column in table

ALTER TABLE table_name

RENAME COLUMN old_name TO new_name;

Alter table tab_name **change column old_column new_column** data_type;

# Drop column in table

ALTER TABLE table_name

DROP COLUMN column_name;

# Rename table

ALTER TABLE table_name

RENAME TO new_table_name;

# Add constraint

ALTER TABLE table_name

ADD  constraint_name

(column_name1,..);

Note:

contraint_name -> unique , primary …..

# Add constraint

ALTER TABLE table_name

ADD PRIMARY KEY (column1, column2, ... column_n);

# Example

- ALTER TABLE supplier ADD PRIMARY KEY (supplier_id);

# Add constraint

```
ALTER TABLE table_name
ADD CONSTRAINT unique_constraint_name
UNIQUE(column_name1, column_nam2);
```

# Drop Primary Key

ALTER TABLE table_name

DROP  constraint_name;

# Example

ALTER TABLE supplier

DROP primary key

# Disable/ Enable Primary Key

ALTER TABLE table_name

DISABLE CONSTRAINT constraint_name;

# DEFAULT

```
CREATE TABLE demo_tbl
(
salary decimal(8,2) DEFAULT 9500,
hire_date DATE DEFAULT “2011-03-1”
);
```